Prog-Py

FS2026 – Prof. Selina Rea Malacarne

Autoren: Mirko Ratti
https://github.com/mirkoratti/FS2026

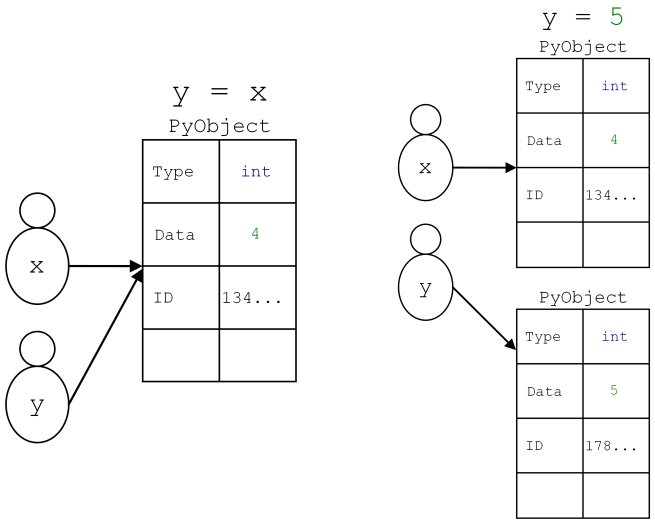
V1.0.20260515

1 Python Overview

- **Origins:** Developed by Guido van Rossum with a focus on simplicity and power.
- **Standard Library:** Known for the "batteries included"philosophy, meaning it comes with an extensive built-in library.
- **Ecosystem:** For engineering tasks, Python replaces Matlab toolboxes with specialized packages like **NumPy** (numerical data), **pandas** (data processing), and **matplotlib** (visualization).

2 Variables and Objects

In Python, everything is an object. Variables are not containers but **references** (pointers) bound to objects.



2.1 Built-in Data Types

Category	Data Type	Mutability	False Value
None	`NoneType`	Immutable	`None`
Numeric	`int`	Immutable	`0`
	`float`	Immutable	`0.0`
	`bool`	Immutable	`False`
	`complex`	Immutable	`0 + 0j`
Sequence	`str`	Immutable	`"`
	`list`	Mutable	`[]`
	`tuple`	Immutable	`()`
	`bytes`	Immutable	`b"`
	`bytearray`	Mutable	`bytearray(b")`
Sets	`set`	Mutable	`set()`
	`frozenset`	Immutable	`frozenset()`
Associative	`dict`	Mutable	`{}`

2.2 Mutability

- **Immutable:** Objects that cannot be modified after creation (e.g., `int`, `float`, `str`, `tuple`). Changing the value of an immutable variable actually creates a new object in memory.
- **Mutable:** Objects that can be modified in-place (e.g., `list`, `set`, `dict`).

The memory address of an object, can be visuasalized using the `id()` function.
It returns a unique integer for the given object.

3 Namespaces and Scope

A namespace is a system (implemented as a dictionary) that ensures all names in a program are unique and can be used without conflict.

- **Built-in:** Contains all integrated symbols available in Python (e.g., `print()`).
- **Global:** Symbols defined at the top level of a module or script.
- **Local:** Created for specific scopes, such as within a function.
- **LEGB Rule:** Python searches for names from the inside out: **L**ocal → **G**lobal → **B**uilt-in.

Built-in symbols can be overwritten by the user, however, it is not recommended.

4 Arithmetic and Magic Methods

- **Special Operators:** `//` (integer division), `%` (modulo/remainder), and `**` (exponentiation).
- **Syntactic Sugar:** Operators are just a "friendly"way to call internal methods. For example, `x + y` is internally executed as `x.__add__(y)`.

Operator	Dunder Method	Description
`x + y`	`x.__add__(y)`	Sum
`x - y`	`x.__sub__(y)`	Difference
`x * y`	`x.__mul__(y)`	Product
`x / y`	`x.__truediv__(y)`	Quotient
`x // y`	`x.__floordiv__(y)`	Floored quotient[1]
`x % y`	`x.__mod__(y)`	Remainder (Modulo)[1]
`x ** y`	`x.__pow__(y)`	Exponentiation (x^y)
`abs(x)`	`x.__abs__()`	Absolute value
`-x`	`x.__neg__()`	Negative sign

[1] Operator not defined for the data type `complex`

Code Example

Ecco come Python interpreta gli operatori dietro le quinte:

```python
# Standard operator usage
result_a = 5 + 5

# Behind the scenes (Magic Method)
result_b = (5).__add__(5)

print(result_a == result_b) # Output: True
```

5 Conditional Branching

- **Basic Structures**: `if`, `elif`, and `else` execute code blocks based on conditions. Consistent indentation is required.
- **Match-case**: Introduced in Python 3.10 to recognize specific patterns and data structures.

```python
x = 5

if x == 5:
    print("5")
else:
    print("Not 5")

match x:
    case 5:
        print("5")
    case 4:
        print("4")
    case _:
        print("Default case")
```

5.1 Boolean Expressions and Operators

- **Walrus Operator (`:=`)**: Assigns and returns a value within an expression.
- **Ternary Operator**: Compact syntax for simple single-line conditional assignments.
- **`==` vs `is`**:
  - `==` checks value equality;
  - `is` checks object identity in memory (ideal for checking `None`).
- **`in` Operator**: Checks presence in objects implementing `__contains__()` (Only usable on iterable Objects: lists, strings, dicts).

```python
# Walrus Operator (:=)
if (a := int(input("Enter a number: "))) > 3:
    print("Number is greater than 3.")

# Ternary Operator
b = 5
a = b if b > 3 else 0

# == vs is
my_list = [1, 2, 3]
print(my_list == [1, 2, 3])  # True (values are identical)
print(my_list is [1, 2, 3])  # False (different objects in memory)

# in Operator
fruits = ["banana", "apple"]
print("apple" in fruits)     # True
```

5.2 Exception Handling

- **LBYL (Look Before You Leap)**: A defensive style where you explicitly check for conditions (`if os.path.exists`) before making a call. This can be prone to "race conditions"where the state changes between the check and the action.
- **EAFP (Easier to Ask Forgiveness than Permission)**: The 'Pythonic' approach. Assume the operation will succeed and wrap it in a `try/except` block. This is generally cleaner and more robust in multi-threaded environments.
- **Catching Exceptions**: Always catch specific exceptions (e.g., `FileNotFoundError`) rather than generic ones. Use the `as` keyword to inspect the exception object.
- **Flow Control**:
  - `else`: Runs only if the `try` block succeeded without errors.
  - `finally`: Always runs (e.g., for closing file handles), regardless of whether an exception was raised or caught.

```python
# EAFP: try, except, else, finally
try:
    x = int(input("Enter a number other than zero: "))
    y = 1 / x
except ValueError:
    print('Oops. Not a valid integer.')
except ZeroDivisionError as e:
```

```python
    print('Oops! ' + str(e))
else:
    print(f"Success! Result: {y}")
finally:
    print("Always executed.")

# LBYL
inp = input("Enter a number other than zero: ")

if(inp.isdecimal()):
    x = int(inp)
    if x != 0:
        print(1/x)
    else:
        print("Wrong Input!")
else:
    print("Wrong Input!")
```

6 Iterations

6.1 Slicing

- Syntax: `part = x[start:end:step]`.
- Start is inclusive (default 0).
- End is exclusive (default end index).
- Step is optional (default 1). A negative step indicates reverse order.

```python
data = [0, 1, 2, 3, 4, 5.0, [1,2,3]]
part = data[::-1] # Reverse order
```

6.2 For-loops

- Internally calls `iter()` to return an iterator.
- Uses the `next(iterator)` method to get the next element until a `StopIteration` exception is raised.

```python
for value in iterable:
    expression1
```

6.2.1 Loop-control statements

- **break**: Terminates a loop before completion.
- **continue**: Skips the remaining code in the loop for the current iteration.
- **else keyword**: Executed when the loop ends normally, meaning it has not encountered a `break` statement.

6.2.2 Dictionaries

```python
for key in data:                # Iterates keys
for value in data.values():     # Iterates values
for key, value in data.items(): # Iterates both keys and values
```

6.2.3 Enumerate

- Used when a counter is needed while iterating through elements.

```python
for i, fruit in enumerate(fruits, start=10):
    print(i, fruit)
```

6.2.4 Zip

- Iterates over multiple iterables at the same time.
- **Objects of different lengths**: The `zip()` function stops as soon as the shortest iterable is exhausted. Any remaining elements in the longer iterables are simply ignored.

```python
colors = ["yellow", "red", "yellow", "blue", "green"] # Length 5
fruits = ["banana", "apple", "lemon", "blueberry"]    # Length 4

# The loop will only run 4 times
for color, fruit in zip(colors, fruits):
    print(color, fruit)
```

6.3 Comprehensions

- **Base Syntax**: `[expression for item in iterable]`.
- **Nested for loops**: Evaluated from left to right.
- **Can also call functions**: Built-in functions, like `any()`, can be used.
- **Conditional Syntax**: The position of the `if` statement completely changes its behavior.
  - **Filtering (Only `if`)**: Must be placed at the **END**. It filters elements out of the iterable.
  - **Modifying (`if/else`)**: Must be placed at the **BEGINNING** (before the `for` loop). It changes the value of the element based on the condition but does not filter it out.

```python
# 1. Nested loops
# Syntax: [expression for outer_item in outer_iterable for inner_item in inner_iterable]
fields = [b + str(n) for b in letters for n in numbers]

# Normal form equivalent
for b in letters:
    for n in numbers:
        b + str(n)

# 2. ONLY IF (Filtering) -> Placed at the END
# Syntax: [expression for item in iterable if condition]
filtered_values = [item**2 for item in range(1, 11) if (item**2) % 4 != 0]

# 3. IF/ELSE (Modifying) -> Placed at the BEGINNING
# Syntax: [expression_if_true if condition else expression_if_false for item in iterable]
modified_values = [item**2 if (item**2) % 4 != 0 else 0 for item in range(1, 11)]
```

7 Built in methods

7.1 Lists

Methods `sort()` and `sorted()`

- `sort()`: Modifies the list in-place. Allows a `reverse=True` parameter for descending order.
- `sorted()`: Returns a new sorted list and can be applied to all iterable objects.
- **The key Argument**: Allows specifying a custom sorting function. It accepts built-in functions (e.g., `str`) or anonymous *lambda* functions for inline sorting criteria.
- **Algorithm Stability and Multiple Priorities**: Python utilizes *Timsort*, a stable sorting algorithm. Stability ensures that when two elements share the same sorting key, they retain their original relative order. To sort by multiple criteria, you can 'stack' consecutive sorts: you must apply the *least* priority sort first, and the *main* priority filter last. Alternatively, you can use a single lambda function that returns a tuple representing the hierarchy of priorities.

```python
data_list = [(1, "D"), (3, "B"), (2, "B"), (5, "E"), (4, "A"), (6, "D")]

# Approach 1: Stacking sorts (exploiting Timsort stability)
# The least priority sort goes first
data_list.sort(key=lambda x: str(x[0]))
# The main priority filter goes last
data_list.sort(key=lambda x: str(x[1]))

# Approach 2: Single sort with a tuple hierarchy
# Equivalent to the stacked approach above
data_list.sort(key=lambda x: (str(x[1]), str(x[0])))
```

Indirect Sorting (Sorting Indices) In many algorithms, it is highly efficient to sort an array of indices rather than moving the actual data elements around. By leveraging a lambda function, we can sort a standard sequence of indices based on the values they point to in the original list.

```python
original = ["F", "B", "C", "G", "A", "D", "E"]
# Generate an array of indices: [0, 1, 2, 3, 4, 5, 6]
idxs = list(range(len(original)))

# Sort the indices based on the alphabetical value of the original elements
sorted_idxs = sorted(idxs, key=lambda i: original[i])
# Result: [4, 1, 2, 5, 6, 0, 3] (e.g., index 4 points to "A")

# Reconstruct the sorted array using the sorted indices
result = [original[i] for i in sorted_idxs]

# Generate a reverse mapping to find where original elements ended up
reverse_idxs = [result.index(char) for char in original]
```

List Slicing Slicing extracts sub-sequences using the `x[start:end:step]` syntax. Besides extraction, slicing is extremely powerful for dynamically modifying lists: it allows replacing an entire area of elements or inserting multiple elements at a specific index simultaneously without overwriting existing data.

```python
data = ['a', 'b', 'c']
# Inserting multiple elements without replacing (at index 3)
data[3:3] = ['#', '$']   # Result: ['a', 'b', 'c', '#', '$']

# Replacing an entire area with new elements
data[1:3] = ['D', 'E']   # Result: ['a', 'D', 'E', '#', '$']
```

7.2 Shallow and Deep Copy

Variable assignment merely binds a name to an object; it does not create a copy.

- **Shallow Copy**: Functions like `list()`, `dict()`, or `.copy()` create a new object only at the top level. Subordinate nested objects are only referenced, meaning modifications to mutable nested objects will affect both the original and the copy.
- **Deep Copy**: Creates a true, recursive copy of the object and all its subordinates.

```python
import copy
list1 = [[1, 2], [3, 4]]
list_shallow = list1.copy()
list_deep = copy.deepcopy(list1)
```

7.3 Strings

Most Used Methods

- **Splitting and Cleaning**: `split(sep)` splits by a separator. `splitlines()` splits at line breaks. `strip()`, `lstrip()` remove leading/trailing characters.
- **Searching and Replacing**: `find(sub)`. `count(sub)`. `replace(old, new)`.
- **Casing**: `lower()`. `upper()`. `capitalize()`. `title()`.
- **Testing**: `isdigit()`. `isalpha()`. `isalnum()`. `islower()`. `isupper()`.

join() and Generators The `join()` method combines all string elements of an iterable into a single string, using the calling string as a separator. It is highly efficient when combined with generator expressions.

```
z = [(10, "Matthias"), (20, "Annabelle")]
# Joining elements using a generator expression
ranking = "\n".join(f"{name} ({points} Points)" for points,
    name in z)
```

F-Strings Formatting Formatting logic follows the syntax: `{<f_expression> [-"] [!<conversion>] [:<format_spec>]}`. Self-documenting expressions can be created by adding an equal sign = after the variable name.

Parameter	Description	Example	Output
Presentation Type (<type>)			
b	Binary Integer	f'{10:b}'	'1010'
c	Single Character	f'{65:c}'	'A'
d	Decimal Integer	f'{75:d}'	'75'
e or E	Exponential Form	f'{0.001:e}'	'1.000000e-03'
f or F	Float	f'{157:f}'	'157.000000'
g or G	Float or Exponential (based on size)	f'{15723823:G}'	'1.57238E+07'
o	Octal Integer	f'{16:o}'	'20'
s	String	f'{"Hallo:s}'	'Hallo'
x or X	Hexadecimal Integer	f'{10:X}'	'A'
%	Percentage	f'{1:%}'	'100.000000%'
Alignment (<align>)			
<	Left-aligned	f'{12:<5}'	'12 '
>	Right-aligned	f'{12:>5}'	' 12'
^	Centered	f'{12:^5}'	' 12 '
=	Sign forced to the far left	f'{-12:=5}'	'- 12'
Fill Character (<fill>)			
char	Any character (used with align)	f'{12:_>5}'	'___12'
Sign (<sign>)			
+	Forces sign for positive/negative	f'{123:+}'	'+123'
-	Sign only for negative numbers	f'{-123:-}'	'-123'
Grouping (<group>)			
, or _	Thousands separator	f'{1000:,}'	'1,000'
Width and Precision (<width>.<precision>)			
width	Minimum output width	f'{12:5}'	' 12'
.precision	Decimal places (for floats)	f'{3.14159:.2f}'	'3.14'

With # is possible to show the alternative output format (0xA10, 0b01101)

7.4 Dictionaries

Most Used Methods

- `get(key, default)`: Returns the value for a key, or a default value (like `None`) if the key does not exist.
- `keys()`, `values()`, `items()`: Return views of the dictionary's keys, values, and key-value pairs, respectively.
- `setdefault(key, default)`: Returns the value of a key; if absent, it inserts the key with the specified default value.
- `pop(key)` and `popitem()`: Remove a specific key-value pair or the last added pair.
- `clear()`: Removes all key-value pairs.

7.5 File Handling

Files are handled using the `open()` function, which accepts a filename and a mode (e.g., `'r'` for reading, `'w'` for writing, `'a'` for appending). Specifying `encoding='utf-8'` is strongly recommended to prevent cross-platform character interpretation issues.
The `with` statement provides a Pythonic context manager that ensures the file is automatically closed upon exiting the block. The `pathlib` module allows for robust, object-oriented path manipulations using the `/` operator, ensuring seamless execution across Unix/macOS environments.

```
import pathlib

# Composing a path object utilizing the / operator
file_path = pathlib.Path.cwd() / 'recipes' / "output.txt"

# Writing files safely using the 'with' context manager
with open(file_path, "w", encoding="utf-8") as f:
    f.write("String data\n")
    f.writelines(["Line 2\n", "Line 3\n"])

# Reading files line by line directly via the file object
with open(file_path, "r", encoding="utf-8") as f:
    for line in f:
        print(line.strip())
```

8 Advanced Unpacking and Functions

8.1 Tuple Packing and Unpacking

Python allows for the grouping and distribution of objects through packing and unpacking mechanisms.

- **Packing:** Multiple individual objects are grouped into a single tuple.
- **Unpacking:** Items within an iterable are assigned to individual variables. This requires the number of variables to match the number of elements unless extended unpacking is used.

```
# Packing
employee = 'Peter', 'Muller' # Results in ('Peter', 'Muller')

# Unpacking
first, last = employee

# Multiple Assignment (combined packing/unpacking)
x, y, z = 5, 10, 15
```

8.1.1 Extended Iterable Unpacking

Unpacking works with any iterable (strings, lists, sets) and supports specific operators to handle variable lengths or unwanted values.

- **Rest Operator (*):** Captures remaining elements into a list.
- **Placeholder (_):** Used by convention to ignore specific values during unpacking.

```
# Unpacking with rest
first, last, *address, birth_date = ("Peter", "Muller", "
    Street", 10, "14.04.1977")
# address becomes ["Street", 10]

# Ignoring values
first, last, _, birth_date = employee_data

# Unpacking in nested structures
x = [1, 2, (3, [4, 5])]
a, b, (c, (d, e)) = x
```

8.1.2 Iterable Expansion

The * and ** operators allow unpacking iterables within other collection literals.

```
list1 = [1, 2]
list2 = [*list1, 3, 4] # [1, 2, 3, 4]

dict1 = {"A": 1, "B": 2}
dict2 = {"C": 3, **dict1} # {'C': 3, 'A': 1, 'B': 2}
```

8.2 Functions

Functions are defined using the `def` keyword. They establish a local scope and can return objects to the caller.

- **Parameters:** Placeholders defined in the function signature.
- **Arguments:** Actual values passed during the call.
- **Return Value:** Functions return `None` by default. Multiple values can be returned as a tuple.

8.2.1 Argument Types and Constraints

- **Positional vs. Keyword:** Positional arguments must precede keyword arguments in a call.
- **Default Parameters:** Parameters with default values must follow those without.
- **Variadic Arguments (*args, **kwargs):** `*args` collects extra positional arguments into a **tuple**; `**kwargs` collects extra keyword arguments into a dictionary.

8.2.2 Argument Enforcement

Developers can force specific call styles using `/` and `*`.

```
def constrained_func(pos_only, /, pos_or_kw, *, kw_only):
    pass

# / : All arguments before must be positional-only.
# * : All arguments after must be keyword-only.
```

8.3 Docstrings

Documentation strings should follow the function definition to describe its behavior. They can be retrieved via `help(function_name)` or the `__doc__` attribute.

```
def add(a, b):
    """Return the sum of a and b."""
    return a + b
```

8.4 Lambda Functions

Lambdas are anonymous, single-expression functions. They are primarily used as "key"functions for sorting or filtering.
Syntax: `lambda parameters: expression`

```
# Sorting a list of strings by the numeric part
articles = ['art1', 'art20', 'art3']
sorted_list = sorted(articles, key=lambda x: int(x[3:]))
```

9 Object-Oriented Programming (OOP)

The fundamental principle of OOP is the encapsulation of data (attributes) and logic (methods) into objects. This structure ensures data integrity by restricting direct external manipulation.

9.1 Class Definition and Instantiation

A class serves as a formal blueprint. Per PEP 8, class names follow the `CapWords` convention. Objects are specific instances created from these blueprints.

```
class Cat:
    """Formal structure of a cat."""
    pass

# Instantiation
garfield = Cat()
```

9.2 Attributes and State Integrity

Attributes store data within an object. Python distinguishes between Class and Instance variables based on their scope and definition.

9.2.1 Class vs. Instance Variables

- **Class Variables:** Defined directly within the class body. They are shared across all instances.
- **Instance Variables:** Defined using `self`. They are unique to each instance.

Critical Warning: Never modify a class variable through an instance (e.g., `instance.class_var = value`). This creates a new instance attribute that shadows the class variable for that specific object.
Always update class variables using the class name: `ClassName.class_var = value`.

9.2.2 Initialization and Dynamic Attributes

It is highly recommended to define **all** instance variables within the `__init__` method.

- **Avoid 'On-the-fly' attributes:** It makes the object's structure unpredictable and harder to debug.
- **Defined State:** Ensuring all attributes are initialized in the constructor

9.3 Method Attributes and self

Methods are functions defined within a class. The first parameter is always `self`, which represents the instance calling the method. Python binds the instance to this parameter automatically during the call.

```
    account1.deposit(100)
    # In background will this executed
    Class_name.deposit(self=account1, amount=100)
```

9.4 Data Abstraction and Encapsulation

Encapsulation hides implementation details, exposing only necessary information through defined interfaces. Python uses naming conventions to signal the intended visibility of attributes:

Category	Convention	Description
Public	`name`	Accessible from anywhere.
Protected	`_name`	Undesirable to access externally; used for inheritance.
Private	`__name`	Hidden via name mangling; not directly accessible.

A private variable is `my_object._MyClass__private_var`

9.5 Properties

The property object is created using the signature:
`var = property(fget=None, fset=None, fdel=None, doc=None)`

```
class BankAccount:
    def __init__(self, balance):
        self._balance = balance

    def _get_balance(self):
        print("Getter called")
        return self._balance

    def _set_balance(self, value):
        print("Setter called")
        self._balance = value

    # Background implementation of the property
    balance = property(_get_balance, _set_balance)
```

9.5.1 Decorators

The same could be achieved using decorators:

```
class BankAccount:
    def __init__(self, balance):
        self._balance = balance

    @property
    def balance(self):
        """Getter: returns the balance."""
        print("Balance requested.")
        return self._balance

    @balance.setter
    def balance(self, value):
        """Setter: updates the balance."""
        if value >= 0:
            self._balance = value
        else:
            print("Invalid amount.")
```

10 Magic Methods and Custom Behavior

10.1 Object Representation

Magic methods (or dunder methods) allow custom classes to interact with built-in Python functions and operators.

- `__str__(self)`: Returns a user-friendly string representation (used by `print()`).
- `__repr__(self)`: Returns an unambiguous string representation, ideally used for debugging or recreating the object.

```
class Book:
    def __init__(self, title, author):
        self.title = title
        self.author = author

    def __str__(self):
        return f"'{self.title}' by {self.author}"

    def __repr__(self):
        return f"Book(title='{self.title}', author='{self.author}')"
```

10.2 Emulating Container Types

Classes can behave like sequences (lists) or mappings (dictionaries) by implementing specific methods:

- `__len__(self)`: Returns the number of items.
- `__getitem__(self, key)`: Defines behavior for indexing (`obj[key]`).
- `__setitem__(self, key, value)`: Defines behavior for item assignment (`obj[key] = value`).

```
class Inventory:
    def __init__(self):
        self._items = {}

    def __setitem__(self, key, value):
        self._items[key] = value

    def __getitem__(self, key):
        return self._items.get(key, "Item not found")

    def __len__(self):
        return len(self._items)
```

10.3 Context Managers

Context managers ensure resources (like files or network connections) are properly managed using the `with` statement. This requires the implementation of two methods:

- `__enter__(self)`: Executed at the start of the `with` block. It returns the resource.
- `__exit__(self, exc_type, exc_val, tb)`: Executed at the end, even if an exception occurs.

```
class FileManager:
    def __init__(self, filename, mode):
        self.filename = filename
        self.mode = mode

    def __enter__(self):
        self.file = open(self.filename, self.mode)
        return self.file

    def __exit__(self, exc_type, exc_val, traceback):
        self.file.close()

with FileManager("data.txt", "w") as f:
    f.write("Structured data")
```

10.4 Reflected Arithmetic Methods (The 'r' Prefix)

Python distinguishes between an operation where an object is on the left-hand side (LHS) and one where it is on the right-hand side (RHS). This is crucial for interoperability with built-in types (like `int` or `float`).

- **`__add__(self, other)`**: Called for `self + other`.
- **`__radd__(self, other)`**: Called for `other + self` only if the LHS object does not define `__add__` for the specific type of `self`.

```
class Price:
    def __init__(self, amount):
        self.amount = amount

    def __add__(self, other):
        # Handles: Price(10) + 5
        val = other.amount if isinstance(other, Price) else other
        return Price(self.amount + val)

    def __radd__(self, other):
```

```
        # Handles: 5 + Price(10)
        # Re-uses __add__ since addition is commutative
        return self.__add__(other)
```

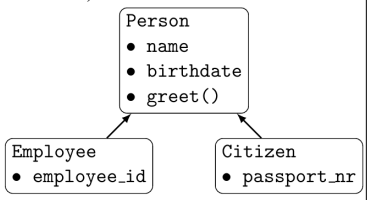
10.5 The NotImplemented Singleton

In Python, `NotImplemented` è un oggetto speciale (un singleton) restituito dai metodi binari (come `__add__`, `__sub__`, `__eq__`, ecc.) per indicare che l'operazione non è implementata per il tipo di dato dell'altro operando.
Importante: Non deve essere confuso con `NotImplementedError`, che è un'eccezione. `NotImplemented` è un valore di ritorno che segnala a Python di tentare una strategia alternativa.

11 Inheritance and Polymorphism

Inheritance enables a new class (subclass or derived class) to adopt the properties and methods of an existing class (superclass or base class).

- code reuse
- modularity
- and the establishment of an "is-a" relationship.



Polymorphism allows subclasses to define methods with the same name as those in the superclass, effectively overriding them. The specific method invoked depends on the actual object type at runtime, increasing interface flexibility.

11.0.1 The super() Function

The `super()` function provides access to methods and attributes of the superclass. It is primarily used within the subclass's `__init__` method to initialize inherited attributes, ensuring the proper setup of the base object state.

```
class Person:
    def __init__(self, name):
        self.name = name

class Employee(Person):
    def __init__(self, name, employee_id):
        super().__init__(name) # Initialize superclass
     attributes
        self.employee_id = employee_id
```

11.0.2 Abstract Classes

Abstract classes serve as templates and cannot be instantiated directly. They define abstract methods that must be implemented by derived classes, enforcing a consistent interface. This requires the `abc` module.

```
from abc import ABC, abstractmethod

class Vehicle(ABC):
    @abstractmethod
    def accelerate(self):
        pass

class Car(Vehicle):
    def accelerate(self):
        print('The car accelerates')
```

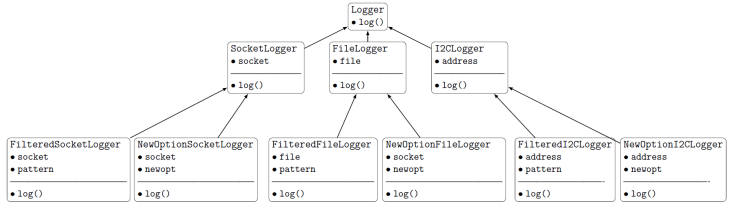
11.1 Design Principles (SOLID)

When constructing inheritance hierarchies, adhering to design principles ensures flexibility and maintainability:

- **S**ingle Responsibility: A class should have only one responsibility and be responsible for exactly one thing.
- **O**pen-Closed: Classes should be open for extension (e.g., via inheritance) but closed for modification.
- **L**iskov Substitution: Objects of a superclass should be replaceable with objects of a subclass without affecting program correctness.
- **I**nterface Segregation: Clients should not be forced to depend on unused methods.
- **D**ependency Inversion: High-level modules should not depend on low-level modules; both should depend on abstractions.

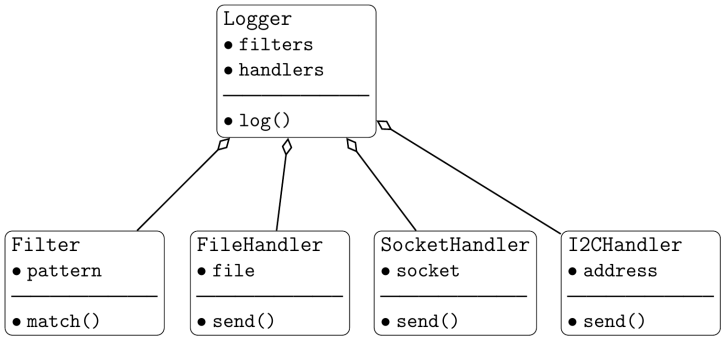
11.2 Aggregation and Composition

Overusing inheritance can lead to a "class explosion" problem, significantly increasing complexity when combining multiple traits or options.



Aggregation and composition offer structural alternatives by composing **classes based on dependencies rather than strict hierarchies.**

- **Composition (is-a-part-of):** Implies a strong lifecycle dependency. The parent object owns and controls the lifetime of the contained objects. If the parent is destroyed, its parts are also destroyed (e.g., a Car and its Engine).
- **Aggregation (has-a):** Implies a loose lifecycle dependency. Contained objects can exist independently of the parent object and can be dynamically added or removed (e.g., a Library and its Books).




```
class Filter:
    def __init__(self, pattern):
        self.pattern = pattern

class Logger:
    # Aggregation: Logger "has" filters and handlers.
    # The components can exist independently of the Logger
     instance.
    def __init__(self, filters, handlers):
        self.filters: list[Filter] = filters
        self.handlers: list[Handlers] = handlers
```

12 NumPy

12.1 Introduction

NumPy (Numerical Python) is a library for performing mathematical functions and operations on large multidimensional arrays. It provides:

- **Speed**: Utilizes optimized C code, replacing slow standard Python `for` loops.
- **Vectorization**: Array-based operations are executed element-wise automatically.
- **Broadcasting**: Implicitly expands arrays of different shapes during arithmetic calculations.

12.2 Creation of n-dimensional Arrays (ndarray)

The `ndarray` is a homogeneous data structure where each dimension is called an *axis*.

- `arr.ndim`: Number of dimensions.
- `arr.shape`: Tuple representing the size of each dimension. (rows, cols)
- `arr.dtype`: Data type of the array elements (e.g., `np.float64`, `np.int32`).
- `arr.astype(dtype)`: Returns a copy of the array cast to a specific type.

12.2.1 Array Creation Methods

```
arr = np.array([1, 2, 3])# Basic 1D array
np.ones((3, 3))          # 3x3 array of 1s
np.zeros_like(arr)       # Array of 0s with the shape of 'arr'
np.full((3, 2), 10)      # 3x2 array filled with 10
np.eye(5)                # 5x5 Identity matrix
np.linspace(0, 1, 10)    # 10 evenly spaced values from 0 to 1
np.arange(0, 4, 0.5)     # Values from 0 to 4 (exclusive),
     step 0.5
np.random.randn(3, 2)    # Standard normally distributed
     random numbers
np.random.randint(0, 100, (3))  # 1D arr with 3 Uniformly
     distributed integers between 0-100
```

Funktion	Resultat
`np.ones((2,2))`	`array([[1., 1.], [1., 1.]])`
`np.ones_like(arr1)`	`array([1, 1, 1])`
`np.zeros((2,2))`	`array([[0., 0.], [0., 0.]])`
`np.zeros_like(arr1)`	`array([0, 0, 0])`
`np.full((2,2), 7.0)`	`array([[7., 7.], [7., 7.]])`
`np.full_like(arr1, 7)`	`array([7, 7, 7])`
`np.eye(2)`	`array([[1., 0.], [0., 1.]])`
`np.linspace(0, 1, 5)`	`array([0., 0.25, 0.5, 0.75, 1.])`
`np.arange(0, 4, 0.5)`	`array([0, 0.5, 1, 1.5, 2, 2.5, 3, 3.5])`
`np.logspace(0, 1, 4)`	`array([1., 2.1544, 4.6416, 10.])`
`np.random.randn(3)`	`array([0.7576, 0.0135, -0.8934])`
`np.random.randint(0,10,3)`	`array([0, 5, 4])`

In `arange` è possibile inserire uno `step` negativo, per creare un array decrescente.
Attenzione al fatto che `end` è escluso dall'array, se si vuole includere: (`end` ± `step`)

12.3 Element Access

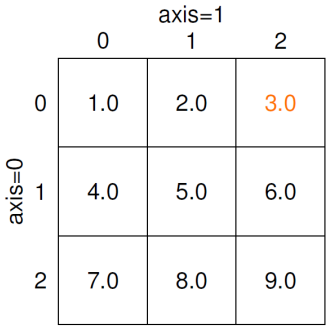
- **Indexing**: Axes are separated by commas within brackets (e.g., `arr[row, col]`).
- **Slicing**: Follows the `[start:end:step]` syntax across multiple axes. Slicing returns a **view**; modifications affect the original array. Use `.copy()` to create an independent duplicate. END not included!
- **Boolean Slicing**: Extracts subsets where a condition is `True` (e.g., `arr[arr > 5]`).
- **Unpacking**: A matrix can be unpacked directly into its components along the first axis: `row1, row2 = np_matrix`

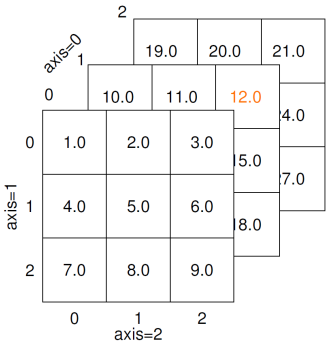
Examples:

arr	Ausdruck	Shape	Resultat
1 2 3 4 5 6 7 8 9	`arr[:2, 1:]`	(2, 2)	`array([[2, 3], [5, 6]])`

1 2 3
4 5 6
7 8 9
	`arr[2]`
	`arr[2, :]`
	`arr[2:, :]`
		(3,)
		(3,)
		(1, 3)
			`array([7, 8, 9])`
			`array([7, 8, 9])`
			`array([[7, 8, 9]])`

1 2 3 4 5 6 7 8 9	`arr[:, 2]` `arr[:, 2:]`	(3,) (3, 1)	`array([3, 6, 9])` `array([[3], [6], [9]])`
1 2 3 4 5 6 7 8 9	`arr[1, :2]` `arr[1:2, :2]`	(2,) (1, 2)	`array([4, 5])` `array([[4, 5]])`





The axes will always change when adding/removing dimesnions

12.4 Broadcasting

To perform operations on arrays of different shapes, NumPy automatically broadcasts the smaller array:

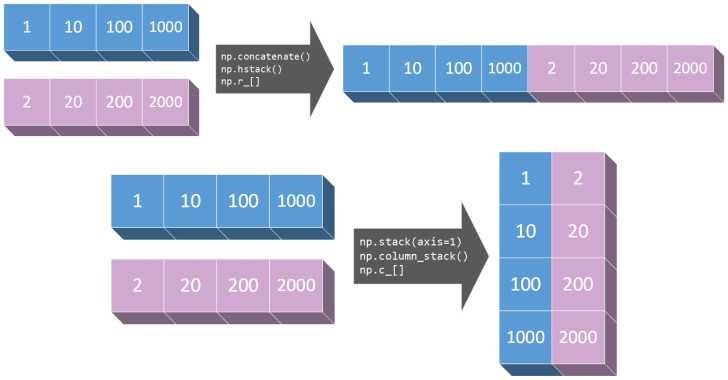
1. Dimensions are inserted on the left until the number of dimensions (`ndim`) matches.
2. Dimensions of size 1 are stretched to match the corresponding dimension of the other array.
3. If sizes differ and neither is 1, a `ValueError` is raised.

12.5 Array Manipulation

12.5.1 Concatenation and Stacking

```
np.concatenate([a, b], axis=0) # Join along an existing axis
np.stack([a, b], axis=0)       # Join along a new axis
np.vstack([a, b])              # Stack vertically (row-wise)
np.hstack([a, b])              # Stack horizontally (col-wise)

# Slice object concatenation (MATLAB-like)
np.r_[1:5, 15, 20:50:10]       # Translates slices to
    concatenated 1D array
np.c_[0:10]                    # Translates slices to
    concatenated 2D columns
```



12.5.2 Dimension Changes

```
arr.reshape(2, -1)      # View with new shape (-1 infers
    dimension)
arr.resize(2, 5)        # In-place shape modification
arr.flatten()           # Returns a 1D COPY
arr.ravel()             # Returns a 1D VIEW
arr[np.newaxis, :]      # Increases dimensions
np.tile(arr, (5, 2))    # Repeats the entire array for the
    given size
np.repeat(arr, 3)       # Repeats individual elements
```

- **View vs Copy**: `arr.reshape(-1)` creates a **view**, not a physical copy. Modifying the reshaped array will affect the original.
- **Automatic Reshaping**: Using `-1` allows NumPy to automatically calculate a dimension based on the array's total size.
- **Compatibility Constraint**: Only one dimension can be set to `-1`. For a reshape to be valid, the total number of elements N must be exactly divisible by the product of the specified dimensions.

12.6 Mathematical Functions

NumPy provides vectorized universal functions (ufuncs) such as: `np.exp()`, `np.sin()`, `np.abs()`, `np.mean()`, `np.max()`, `np.std()`, and `np.cumsum()`.

12.7 Concatenation

MATLAB-like Concatenation:

- `np.r_`: Concatenates along the first axis (row-wise). It supports slice objects and individual values.
- *Example:* `x = np.r_[1:5, 15, 20:50:10]` → `[1, 2, 3, 4, 15, 20, 30, 40]`
- `np.c_`: Concatenates along the second axis (column-wise). Ideal for creating column vectors.
- *Example:* `y = np.c_[0:3]` → `[[0], [1], [2]]`

12.8 NumPy Linear Algebra: Multiplication Summary

12.8.1 Scalar and Vector Products

- **np.inner(a, b):** Best for the **Scalar Product** (inner product) of two vectors.
  - *When to use:* Use it when you specifically want the inner product of vectors. It is clearer than `dot()` because it is restricted to this specific operation.
  - *Example:* `v1 = [1, 2]; v2 = [3, 4]` → `1*3 + 2*4 = 11`.
- **np.outer(a, b):** Computes the **Dyadic Product** (outer product).
  - *When to use:* Use it to create a matrix from two vectors (Column Vector × Row Vector).
  - *Example:* `v1 = [1, 2]; v2 = [3, 4]` → `[[3, 4], [6, 8]]`.

12.8.2 Matrix Multiplication: @ vs np.dot()

While both perform standard matrix multiplication for 2D arrays, they behave differently for higher-dimensional tensors ($n > 2$).

- **The @ Operator (np.matmul):** The standard for **Matrix Multiplication**.
  - *Behavior ($n > 2$):* It treats the array as a 'stack' of matrices and multiplies them across their final two dimensions.
  - *Recommendation:* This is the preferred method for standard matrix multiplication in modern Python (3.5+).
- **np.dot(a, b):**
  - *Behavior ($n > 2$):* It calculates the sum of products across the **last** dimension of a and the **penultimate** dimension of b.
  - *Result:* It effectively multiplies the first row of a by EVERY column in each subsequent matrix of b, leading to a much larger resulting shape.

Usually always use the `@` operator

12.8.3 Comparison Example ($n > 2$)

```
a = np.ones((2, 3, 3))
b = np.ones((2, 3, 3))

# Result: (2, 3, 3) -> Multiplies stacks independently
res_matmul = a @ b

# Result: (2, 3, 2, 3) -> Multiplies every row by every column
res_dot = np.dot(a, b)
```

12.8.4 Methods

The `np.linalg` module provides essential tools for matrix computations.

- **np.linalg.solve(A, b)**: Computes the exact solution of a linear matrix equation $Ax = b$.
  - A must be square and of full rank (non-singular).
- **np.linalg.lstsq(A, b, rcond=None)**: Returns the **Least-Squares** solution to a linear matrix equation.
  - *When to use*: When $Ax = b$ has no exact solution
  - *Returns array*: A tuple containing the solution x, residuals, rank of A, and singular values.
- **np.linalg.eig(A)**: Computes the eigenvalues and right eigenvectors of a square array.
  - *Returns*: A tuple `(w, v)` where `w` contains the eigenvalues and `v` the corresponding Matrix with eigenvectors as columns.
- `np.linalg.inv(A)`: Computes the multiplicative inverse of a square matrix.
- `np.linalg.det(A)`: Calculates the determinant of a matrix.
- `np.trace(A)`: Computes the trace, which is the sum of the diagonal elements of an array .
- `np.linalg.matrix_rank(A)`: Returns the matrix rank of an array.
- `np.linalg.norm(R)`: Computes the Euclidean norm (magnitude), defined as the square root of the sum of the squares of the vector's components Formula: $\|p\|_2 = \sqrt{x^2 + y^2 + z^2}$.

Example Usage:

```
A = np.array([[3, 1], [1, 2]])
b = np.array([9, 8])

# Exact solution
x = np.linalg.solve(A, b)

# Eigen-decomposition
eigenvalues, eigenvectors = np.linalg.eig(A)

# Least squares (e.g., for linear regression)
```

```
# rcond=None uses the machine precision default
x_ls, res, rank, s = np.linalg.lstsq(A, b, rcond=None)
```

13 Data Preprocessing and Cleaning

13.1 Resampling

```
# --- DOWNSAMPLING ---
# 1. Slicing: Take every n-th sample (Fast, but loses information)
t_down = t[::10]

# 2. Block Averaging / Mean Pooling: Averages blocks of size n (Low-pass filter)
s_mean = np.mean(s[:end].reshape(-1, 10), axis=-1)

# 3. Smoothing and Decimation: Moving average followed by slicing
kernel = np.ones(20) / 20
s_smooth = np.convolve(s, kernel, mode='same')
s_down = s_smooth[::10]

# --- UPSAMPLING ---
# 1. Zero-Order Hold (ZOH): Repeats samples
s_zoh = np.repeat(s, 4)

# 2. Interpolation: Estimates missing values (Linear or Cubic)
lin_interp = interp1d(t, s, kind='linear', fill_value="extrapolate")
s_lin = lin_interp(t_up)
```

13.2 Data Cleaning & Scaling

```
# --- HANDLING NaNs ---
nan_mask = np.isnan(data)
data_no_nan = data[~nan_mask]                    # Remove NaNs
mean_val = np.nanmean(data)                      # Compute mean ignoring NaNs

data_filled = data.copy()
data_filled[nan_mask] = mean_val                 # Replace NaNs with mean
data_interp = pd.Series(data).interpolate()      # Interpolate missing values

# --- OUTLIERS & CLIPPING ---
clean_data = data[data < 20]                     # Keep values below threshold
clipped_data = np.clip(data, 0, 10)              # Force values into [0, 10] range

# --- FEATURE SCALING ---
# 1. Min-Max Normalization (Rescales to [0, 1])
normalized = (data - np.min(data)) / (np.max(data) - np.min(data))

# 2. Z-score Standardization (Mean = 0, Std = 1)
standardized = (data - np.mean(data)) / np.std(data)
```

13.3 NumPy File Handling

```
# Save array to a text file with specific formatting
x = np.arange(100).reshape(20,5)
np.savetxt("my_data.txt", x, fmt="%2d", delimiter=",")

# Load specific columns from a text file
y = np.loadtxt("my_data.txt", delimiter=",", usecols=(0, 2, 4))
```

14 Data Visualization

Data visualization is crucial because summary statistics can be misleading and the choice of plot directly impacts the drawn conclusions. Matplotlib is a comprehensive library for creating static, animated, and interactive visualizations in Python.

14.1 Matplotlib APIs

There are essentially two approaches to creating plots: the non-object-oriented approach via the functional interface (`pyplot`) and the object-oriented (OOP) approach. The OOP approach is recommended as it offers greater control and customization.

```
import matplotlib.pyplot as plt
import numpy as np

t = np.linspace(0, 1, 100)
s1 = np.sin(2 * np.pi * t)

# Object-Oriented API (Recommended)
fig, ax = plt.subplots(layout="constrained")
ax.plot(t, s1, linestyle='-', marker='.', color="blue", label='Simulation')
ax.set_xlabel('Time (s)')
ax.set_ylabel('Amplitude (V)')
ax.grid(which="both")
ax.legend()
plt.show()
```

14.2 Common Methods and Formatting Arguments

When customizing plots, several key arguments are frequently used across different Matplotlib methods:

- **Figure and Layout (`plt.subplots`, `plt.figure`):**
  - `figsize=(width, height)`: Defines the dimensions of the figure in inches.
  - `layout="constrained"`: Automatically adjusts subplots to fit within the figure area, preventing overlapping elements.
  - `sharex, sharey`: Forces subplots to share the same X or Y axis scale (useful for direct comparisons).
- **Plot Styling (`ax.plot`, `ax.scatter`):**
  - `color` or `c`: Sets the color of the plot elements.
  - `linestyle` or `ls`: Defines the line style (`'-'` solid, `'--'` dashed, `'-.'` dash-dot, `':'` dotted).
  - `linewidth` or `lw`: Sets the stroke width of the line.
  - `marker`: Adds specific symbols to data points.
  - `alpha`: Controls transparency (0.0 is fully transparent, 1.0 is fully opaque).
  - `label`: Assigns a string identifier to the plot trace, which is then used by `ax.legend()`.
- **Text and Labels (`ax.set_title`, `ax.set_xlabel`, `ax.text`):**
  - `fontsize`: Specifies the size of the text elements.
  - `ha`, `va`: Horizontal and vertical alignment for text annotations (e.g., `'center'`, `'left'`, `'top'`, `'bottom'`).
  - `loc`: Position argument, commonly used to place legends (e.g., `ax.legend(loc='upper right')`).

14.3 Common Plot Types

Depending on the data, different plot types are more effective:

- **Line Plots**: Useful for continuous signals. Logarithmic axes can be applied using `loglog()`, `semilogx()`, or `semilogy()`.
- **Step Plots**: Created with `step()`, plotting data with a horizontal baseline connected by vertical lines.
- **Histograms**: Visualizes numerical data distribution using `hist()`.
- **Scatter Plots**: Displays measurement data as points on X and Y axes using `scatter()`. Size and color can represent third and fourth dimensions.
- **3D and Contour Plots**: `contour()` or `contourf()` for 2D cross-sections of a 3D surface, and `plot_surface()` for 3D renderings.

14.4 Subplots and Layouts

When managing multiple plots, `plt.subplots()` creates a figure and an array of axes objects.

```
# Creating a 2x2 grid of subplots sharing X and Y axes
fig, axes = plt.subplots(nrows=2, ncols=2, sharex=True, sharey=True)
axes[0, 0].plot(t, s1)
axes[0, 0].set_title("First Plot")
```

14.5 Library Comparison

- **Matplotlib**: Full control and customization, highly customizable but static.
- **Seaborn**: Built on top of Matplotlib, provides high-level statistical plots with clean default styles and integrates perfectly with Pandas.
- **Plotly**: Web-based plots that allow zooming, hovering, and dynamic updates.

15 Appendix

15.1 Type Conversion: `dict()` and `list()`

- **Tuple to Dict**: The `dict()` function can convert a list of 2-item tuples directly into key-value pairs.
- **Deduplication**: Since dictionary keys must be unique, converting a list of tuples to a `dict` automatically removes duplicate entries (keeping the last evaluated value for a given key).
- **Dict to List**: Converts the deduplicated dictionary back into a list of tuples using `list(dictionary.items())`.

```python
# List of tuples containing duplicate entries
contacts = [
    ('Maria', '022 548 95 26'),
    ('Patrick', '081 445 66 58'),
    ('Patrick', '081 445 66 58') # Duplicate
]

# 1. dict(contacts) removes the duplicate 'Patrick' key
# 2. .items() returns the key-value pairs
# 3. list() converts them back into a list of tuples
data_unique = list(dict(contacts).items())

print(data_unique)
# Output: [('Maria', '022 548 95 26'), ('Patrick', '081 445 66
      58')]
```

15.2 Decorators

Decorators are functions that take another function as an argument to extend its behavior without modifying the original source code. They are denoted by the `@` symbol.

```python
def require_login(func):
    def wrapper(user):
        if user["logged_in"]:
            return func(user)
        print("Access denied.")
    return wrapper

@require_login
def view_dashboard(user):
    print(f"Welcome {user['name']}")
```

15.3 Script Execution

To distinguish between a script being run directly and a script being imported as a module, Python uses the `__name__` variable.

- When run directly: `__name__ == "__main__"`.
- When imported: `__name__` equals the module's filename.

```python
def main():
    # Test code or entry point
    pass

if __name__ == "__main__":
    main()
```

15.4 Distribuzioni Casuali in NumPy

- **np.random.rand**: Genera campioni da una distribuzione **uniforme** nell'intervallo $[0, 1)$. Ogni valore ha la stessa probabilità di essere estratto.
- **np.random.randn**: Genera campioni dalla distribuzione **normal standard** (Gaussiana) con media $\mu = 0$ e varianza $\sigma^2 = 1$.

15.4.1 Trasformazione della Distribuzione Normale

Per scalare e traslare la distribuzione normale standard $\mathcal{N}(0, 1)$ verso una distribuzione personalizzata $\mathcal{N}(\mu, \sigma^2)$, si applica la seguente formula lineare:

$$X = \sigma \cdot \text{np.random.randn}(\dots) + \mu$$

Dove:

- σ (Sigma): Deviazione standard (controlla la "larghezza"della campana).
- μ (Mu): Media (sposta il centro della distribuzione).

15.5 Indexing vs Slicing: Regola Rapida

La differenza dipende dal tipo di selettore utilizzato per l'asse:

- **Intero (Indexing)**: Se usi un numero secco (es. `arr[2]`), la dimensione viene **rimossa** (rank-1).
- **Range (Slicing)**: Se usi i due punti (es. `arr[2:3]`), la dimensione viene **mantenuta** (rank-2).

15.5.1 Esempio Pratico

Dato un array 2D di shape `(3, 3)`:

- `arr[2, :]` → Risultato 1D, shape `(3,)`.
- `arr[2:3, :]` → Risultato 2D, shape `(1, 3)`.

Python – Summary

Built-in Functions

```
abs(x) ➡ num
all(iterable) ➡ bool
any(iterable) ➡ bool
ascii(object) ➡ str
bin(x) ➡ str
bool([x]) ➡ bool
breakpoint(*args, **kws)
bytearray([source[,encoding[,errors]]]) ➡ bytearray
bytes([source[, encoding[, errors]]]) ➡ bytes
callable(object) ➡ bool
chr(i) ➡ str
@classmethod
compile★
complex([real[, imag]]) ➡ complex
delattr(object, name)
dict(**kwarg) ➡ dict
dict(mapping, **kwarg) ➡ dict
dict(iterable, **kwarg) ➡ dict
dir([object]) ➡ list
divmod(a, b) ➡ num
enumerate(iterable, start=0) ➡ enumerate object
eval(expression[, globals[, locals]]) ➡ object
exec(object[, globals[, locals]])
filter(function, iterable)
float([x]) ➡ float
format(value[, format_spec]) ➡ str
frozenset([iterable]) ➡ frozenset
getattr(object, name[, default]) ➡ object
globals() ➡ dict
hasattr(object, name) ➡ bool
hash(object) ➡ int
help([object])
hex(x) ➡ str
id(object) ➡ int
input([prompt]) ➡ str
int([x]) ➡ class
int(x, base=10) ➡ int
isinstance(object, classinfo) ➡ bool
issubclass(class, classinfo) ➡ bool
iter(object[, sentinel]) ➡ iterator
len(s) ➡ int
list([iterable]) ➡ list
locals() ➡ dict
map(function, iterable, ...) ➡ iterator
max(iterable, *[, key, default]) ➡ object
max(arg1, arg2, *args[, key]) ➡ object
min(iterable, *[, key, default]) ➡ object
min(arg1, arg2, *args[, key]) ➡ object
next(iterator[, default]) ➡ object
object ➡ object
oct(x) ➡ str
open★
ord(c) ➡ int
pow(base, exp[, mod]) ➡ num
print★
property★
range(stop) ➡ sequence object
range(start, stop[, step]) ➡ sequence object
repr(object) ➡ str
reversed(seq) ➡ iterator
round(number[, ndigits]) ➡ num
set([iterable]) ➡ set
setattr(object, name, value)
slice(stop) ➡ slice object
slice(start, stop[, step]) ➡ slice object
sorted(iterable, *, key=None, reverse=False) ➡ list
@staticmethod
str(object=) ➡ str
str★
sum(iterable, /, start=0) ➡ num
super([type[, object-or-type]]) ➡ proxy object
tuple([iterable]) ➡ tup
type(object) ➡ type object
type(name, bases, dict) ➡ type
vars([object]) ➡ __dict__ attribute
zip(*iterables) ➡ iterator
__import__★
```

★

```
__import__(name, globals=None, locals=None, fromlist=(), level=0)
compile(source, filename, mode, flags=0, dont_inherit=False, optimize=-1) ➡ AST object
print(*objects, sep=, end="\n", file=sys.stdout, flush=False)
property(fget=None, fset=None, fdel=None, doc=None) ➡ property attribute
str(object=b, encoding="utf-8", errors="strict") ➡ str

open(file, mode="r", buffering=-1, encoding=None, errors=None, newline=None, ...) ➡ file object (file)
mode = "r" | "w" | "x" | "a" | "b" | "t" | "+"
file.close()
file.read(size) ➡ str | bytes
file.readlines(hint) ➡ list
file.writelines(list)
file.readline(size) ➡ str
file.write(str | bytes)
```

Dict Methods

```
dict.clear()
dict.copy() ➡ dict
dict.fromkeys(iterable[, value]) ➡ dict
dict.get(key[, default]) ➡ object
dict.items() ➡ dict_items
dict.keys() ➡ dict_keys
dict.pop(key[, default]) ➡ object
dict.popitem() ➡ tup
dict.setdefault(key[, default]) ➡ object
dict.update([other])
dict.values() ➡ dict_values
```


List Methods

```
list.append(x)
list.clear()
list.copy() ➡ list
list.count(x) ➡ int
list.extend(iterable)
list.index(x[, start[, end]]) ➡ int
list.insert(i, x)
list.pop([i]) ➡ object
list.remove(x)
list.reverse()
list.sort(key=None, reverse=False)
```

String Methods

Testing

```
str.endswith(suffix[, start[, end]]) ➡ bool
str.isalnum() ➡ bool
str.isalpha() ➡ bool
str.isascii() ➡ bool
str.isdecimal() ➡ bool
str.isdigit() ➡ bool
str.isidentifier() ➡ bool
str.islower() ➡ bool
str.isnumeric() ➡ bool
str.isspace() ➡ bool
str.istitle() ➡ bool
str.isupper() ➡ bool
str.isprintable() ➡ bool
str.startswith(prefix[, start[, end]]) ➡ bool
```

Formatting

```
str.center(width[, fillchar]) ➡ str
str.format(*args, **kwargs) ➡ str
str.format_map(mapping) ➡ str
str.rjust(width[, fillchar]) ➡ str
str.ljust(width[, fillchar]) ➡ str
str.zfill(width) ➡ str
```

Transformation

```
str.encode★
str.expandtabs(tabsize=8) ➡ str
str.lstrip([chars]) ➡ str
str.replace(old, new[, count]) ➡ str
str.rstrip([chars]) ➡ str
str.strip([chars]) ➡ str
```

★

```
str.encode(encoding="utf-8", errors="strict") ➡ bytes object
```

Upper and Lower Cases

```
str.capitalize() ➡ str
str.casefold() ➡ str
str.lower() ➡ str
str.swapcase() ➡ str
str.title() ➡ str
str.upper() ➡ str
```

Split & Combine

```
str.join(iterable) ➡ str
str.partition(sep) ➡ tup
str.rpartition(sep) ➡ tup
str.split(sep=None, maxsplit=-1) ➡ list
str.rsplit(sep=None, maxsplit=-1) ➡ list
str.splitlines([keepends]) ➡ list
```

Searching

```
str.count(sub[, start[, end]]) ➡ int
str.find(sub[, start[, end]]) ➡ int
str.index(sub[, start[, end]]) ➡ int
str.rfind(sub[, start[, end]]) ➡ int
str.rindex(sub[, start[, end]]) ➡ int
```

String Formatting

`{<f_expression> ["="] [!<conversion>] [:<format_spec>]}`

conversion	`s`: with `str()`, `r`: with `repr()`, `a`: with `ascii()`,
format_spec	`[[fill]align][sign][#][0][width][group][.precision][type]`
fill	`<any character>` (except "{" and "}")
align	"<" | ">" | "=" | "^"
sign	"+" | "-" | " "
width	positive integer
group	"_" | ","
precision	positive integer
type	"b" | "c" | "d" | "e" | "E" | "f" | "F" | "g" | "G" | "n" | "o" | "s" | "x" | "X" | "%"

Set Methods

```
set.add(elem)
set.clear()
set.copy() ➡ set
set.difference(*others) ➡ set
set.difference_update(*others)
set.discard(elem)
set.intersection(*others) ➡ set
set.intersection_update(*others)
set.isdisjoint(bool)
set.issubset(other) ➡ bool
set.issuperset(other) ➡ bool
set.pop() ➡ object
set.remove(elem)
set.symmetric_difference(other) ➡ set
set.symmetric_difference_update(other)
set.union(*others) ➡ set
set.update(*others)
```

NumPy (`import numpy as np`)

Legende: "➡ ndarray" = copy | "➡ ndarray ◇" = view | "➡ ndarray (◇)" = view/copy

Array Creation

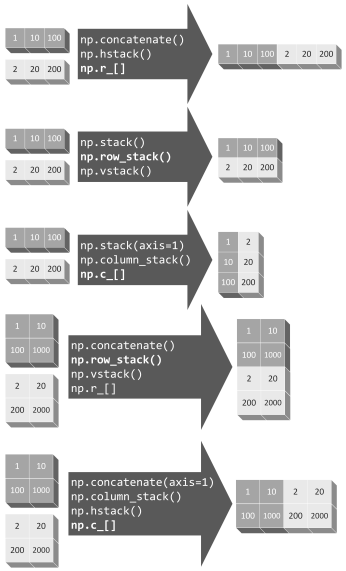
```
np.array(object, dtype=None, *, copy=True, order="K", subok=False, ndmin=0) ➡ ndarray
np.arange([start,] stop[, step,]dtype=None) ➡ ndarray
np.asarray(a[, dtype, order]) ➡ ndarray (◇)
np.copy(a, order="K", subok=False) ➡ ndarray
np.empty(shape, dtype=float, order="C") ➡ int ➡ ndarray
np.empty_like(prototype, dtype=None, order="K", subok=True, shape=None) ➡ ndarray
np.eye(N, M=None, k=0, dtype=<class "float">, order="C") ➡ ndarray
np.full(shape, fill_value, dtype=None, order="C") ➡ ndarray
np.full_like(a, fill_value, dtype=None, order="K", subok=True, shape=None) ➡ ndarray
np.fromfile(file, dtype=float, count=-1, sep=, offset=0) ➡ ndarray
np.fromstring(string, dtype=float, count=-1, sep=) ➡ ndarray
np.identity(n, dtype=None) ➡ ndarray
np.linspace(start, stop, num=50, endpoint=True, retstep=False, dtype=None, axis=0) ➡ ndarray
np.loadtxt(fname, dtype=<class "float">, comments="#", delimiter=None, converters=None, skiprows=0,
    usecols=None, unpack=False, ndmin=0, encoding="bytes", max_rows=None) ➡ ndarray
np.logspace(start, stop, num=50, endpoint=True, base=10.0, dtype=None, axis=0) ➡ ndarray
np.meshgrid(*xi, copy=True, sparse=False, indexing=*y") ➡ ndarray
np.ones(shape, dtype=None, order="C") ➡ ndarray
np.ones_like(a, dtype=None, order="K", subok=True, shape=None) ➡ ndarray
np.random.rand(d0, d1, ..., dn) ➡ ndarray
np.random.randn(d0, d1, ...,dn) ➡ ndarray
np.random.randint(low, high=None, size=None, dtype=int) ➡ ndarray
np.zeros(shape, dtype=float, order="C") ➡ ndarray
np.zeros_like(a, dtype=None, order="K", subok=True, shape=None) ➡ ndarray
```

Array Manipulation

```
numpy.append(arr, values, axis=None) ➡ ndarray
np.array_split(ary, indices_or_sections, axis=0) ➡ list
np.atleast_1d(*arys) ➡ ndarray
np.atleast_2d(*arys) ➡ ndarray
np.atleast_3d(*arys) ➡ ndarray
np.c_[...] ➡ ndarray
np.column_stack(tup) ➡ ndarray
np.concatenate((a1, a2, ...), axis=0, out=None) ➡ ndarray
np.dstack(tup) ➡ ndarray
np.expand_dims(a, axis) ➡ ndarray ◇
np.ndarray.flatten([order]) ➡ ndarray
np.hstack(tup) ➡ ndarray
np.r_[...] ➡ ndarray
np.ravel(a, order="C") ➡ ndarray ➡ array_like (◇)
np.repeat(a, repeats, axis=None) ➡ ndarray
np.reshape(a, newshape, order="C") ➡ ndarray (◇)
np.resize(a, newshape) ➡ ndarray
np.roll(a, shift, axis=None) ➡ ndarray
np.row_stack(tup) ➡ ndarray
np.split(ary, indices_or_sections, axis=0) ➡ list ◇
np.stack(arrays, axis=0, out=None) ➡ ndarray
np.tile(A, reps) ➡ ndarray
np.transpose(a, axes=None) ➡ ndarray
np.ndarray.T ➡ ndarray (◇)
np.unravel_index(indices, shape, order="C") ➡ tup
np.vstack(tup) ➡ ndarray
```

Description	1D→1D	1D→2D	2D→2D	2D→3D
`np.concatenate(arrays, axis=0, out=None)` Join a sequence of arrays along an **existing** axis.	0[1]	–	0/1	–
`np.stack(arrays, axis=0, out=None)` Join a sequence of arrays along a **new** axis.	–	0/1	–	0/1/2
`np.column_stack(tup)` Stack 1-D arrays as columns into a 2-D array.	–	1	1	–
`np.row_stack(tup)` Stack arrays in sequence vertically (row wise).	–	0	0	–
`np.hstack(tup)` Stack arrays in sequence horizontally (col wise).	0	–	1	–
`np.vstack(tup)` Stack arrays in sequence vertically (row wise).	–	0	0	–
`np.c_[]` Translates slice objects to concat. along the second axis.	–	1	1	–
`np.r_[]` Translates slice objects to concat. along the first axis.	0	–	0	–

[1] This number specifies the axis over which the arrays are stacked.



Mathematical Functions

```
np.arccos(x, /, out=None, *, where=True, casting='same_kind', order='K', dtype=None, subok=True[, signature, extobj]) ➡ ndarray
np.arcsin(x, /, out=None, *, where=True, casting='same_kind', order='K', dtype=None, subok=True[, signature, extobj]) ➡ ndarray
np.arctan(x, /, out=None, *, where=True, casting='same_kind', order='K', dtype=None, subok=True[, signature, extobj]) ➡ ndarray
np.arctan2(x1, x2, /, out=None, *, where=True, casting='same_kind', order='K', dtype=None, subok=True[, signature, extobj]) ➡ ndarray
np.ceil(x, /, out=None, *, where=True, casting='same_kind', order='K', dtype=None, subok=True[, signature, extobj]) ➡ ndarray|scalar
np.cos(x, /, out=None, *, where=True, casting='same_kind', order='K', dtype=None, subok=True[, signature, extobj]) ➡ ndarray
np.cumsum(a, axis=None, dtype=None, out=None) ➡ ndarray
np.degrees(x, /, out=None, *, where=True, casting='same_kind', order='K', dtype=None, subok=True[, signature, extobj]) ➡ ndarray
np.diff(a, n=1, axis=-1, prepend=<no value>, append=<no value>) ➡ ndarray
np.exp(x, /, out=None, *, where=True, casting='same_kind', order='K', dtype=None, subok=True[, signature, extobj]) ➡ ndarray|scalar
np.floor(x, /, out=None, *, where=True, casting='same_kind', order='K', dtype=None, subok=True[, signature, extobj]) ➡ ndarray|scalar
np.log(x, /, out=None, *, where=True, casting='same_kind', order='K', dtype=None, subok=True[, signature, extobj]) ➡ ndarray
np.log10(x, /, out=None, *, where=True, casting='same_kind', order='K', dtype=None, subok=True[, signature, extobj]) ➡ ndarray
np.radians(x, /, out=None, *, where=True, casting='same_kind', order='K', dtype=None, subok=True[, signature, extobj]) ➡ ndarray
np.sin(x, /, out=None, *, where=True, casting='same_kind', order='K', dtype=None, subok=True[, signature, extobj]) ➡ ndarray
np.sum(a, axis=None, dtype=None, out=None, keepdims=<no value>, initial=<no value>, where=<no value>) ➡ ndarray
np.sqrt(x, /, out=None, *, where=True, casting='same_kind', order='K', dtype=None, subok=True[, signature, extobj]) ➡ ndarray
np.tan(x, /, out=None, *, where=True, casting='same_kind', order='K', dtype=None, subok=True[, signature, extobj]) ➡ ndarray
```

Linear Algebra

```
np.cross(a, b, axisa=-1, axisb=-1, axisc=-1, axis=None) ➡ ndarray
np.dot(a, b, out=None) ➡ ndarray
np.linalg.det(a) ➡ array_like
np.linalg.eig(a) ➡ ndarray
np.inner(a, b) ➡ ndarray
np.linalg.inv(a) ➡ ndarray
np.matmul(x1, x2, **kwargs) ➡ ndarray
np.linalg.matrix_rank(M, tol=None, hermitian=False) ➡ array_like
np.linalg.norm(x, ord=None, axis=None, keepdims=False) ➡ float|ndarray
np.outer(a, b, out=None) ➡ ndarray
np.linalg.solve(a, b) ➡ ndarray
M @ v ➡ ndarray
np.trace(a, offset=0, axis1=0, axis2=1, dtype=None, out=None) ➡ ndarray
```

Matplotlib (`import matplotlib.pyplot as plt`)

Creation

```
plt.figure(num=None, figsize=None, dpi=None, facecolor=None, edgecolor=None, frameon=True,
    FigureClass=<class "matplotlib.figure.Figure">, clear=False, **kwargs) ➡ Figure
plt.subplot(*args, **kwargs) ➡ AxesSubplot
plt.subplots(nrows=1, ncols=1, *, sharex=False, sharey=False, squeeze=True, subplot_kw=None,
    gridspec_kw=None, **fig_kw) ➡ Figure, axes.Axes
plt.twinx(ax=None) ➡ AxesSubplot
plt.twiny(ax=None) ➡ AxesSubplot
plt.tight_layout(*, pad=1.08, h_pad=None, w_pad=None, rect=None)
plt.savefig(*args, **kwargs)
plt.show(*, block=None)
```

Drawing

```
plt.annotate(text, xy, *args, **kwargs) ➡ Annotation
plt.arrow(x, y, dx, dy, **kwargs) ➡ FancyArrow
plt.contour(*args, data=None, **kwargs) ➡ QuadContourSet
plt.contourf(*args, data=None, **kwargs) ➡ QuadContourSet
plt.loglog(*args, **kwargs) ➡ list of Line2D objects
plt.plot(*args, scalex=True, scaley=True, data=None, **kwargs) ➡ list of Line2D objects
plt.scatter(x, y, s=None, c=None, marker=None, cmap=None, norm=None, vmin=None, vmax=None, alpha=None,
    linewidths=None, verts=<deprecated parameter>, edgecolors=None, *, plotnonfinite=False, data=None,
    **kwargs) ➡ PathCollection
plt.semilogx(*args, **kwargs) ➡ list of Line2D objects
plt.semilogy(*args, **kwargs) ➡ list of Line2D objects
plt.stem(*args, linefmt=None, markerfmt=None, basefmt=None, bottom=0, label=None,
    use_line_collection=True, data=None) ➡ StemContainer
plt.step(x, y, *args, where="pre", data=None, **kwargs) ➡ list of Line2D objects
plt.streamplot(x, y, u, v, density=1, linewidth=None, color=None, cmap=None, norm=None, arrowsize=1,
    arrowstyle="-|>", minlength=0.1, transform=None, zorder=None, start_points=None, maxlength=4.0,
    integration_direction="both", *, data=None) ➡ StreamplotSet
plt.text(x, y, s, fontdict=None, **kwargs) ➡ Text
```

Decoration

```
plt.axis(*args, emit=True, **kwargs) ➡ Annotation
plt.axhline(y=0, xmin=0, xmax=1, **kwargs) ➡ Line2D
plt.axvline(x=0, ymin=0, ymax=1, **kwargs) ➡ Line2D
plt.colorbar(mappable=None, cax=None, ax=None, **kw) ➡ Colorbar
plt.grid(b=None, which="major", axis="both", **kwargs)
plt.legend(*args, **kwargs)
plt.margins(*margins, x=None, y=None, tight=True) ➡ tup
plt.suptitle(t, **kwargs) ➡ Text
plt.title(label, fontdict=None, loc=None, pad=None, *, y=None, **kwargs) ➡ Text
plt.xlabel(xlabel, fontdict=None, labelpad=None, *, loc=None, **kwargs)
plt.xlim(*args, **kwargs) ➡ tup
plt.xticks(ticks=None, labels=None, **kwargs) ➡ tup
plt.ylabel(ylabel, fontdict=None, labelpad=None, *, loc=None, **kwargs)
plt.ylim(*args, **kwargs) ➡ tup
plt.yticks(ticks=None, labels=None, **kwargs) ➡ tup
```

Decoration OOP (The names of the listed OOP methods differ from the non-OOP method names.)

```
Axes.set_title(self, label, fontdict=None, loc=None, pad=None, *, y=None, **kwargs) ➡ Text
Axes.set_xlabel(self, xlabel, fontdict=None, labelpad=None, *, loc=None, **kwargs)
Axes.set_xlim(self, left=None, right=None, emit=True, auto=False, *, xmin=None, xmax=None) ➡ tup
Axes.set_xticks(self, ticks, *, minor=False) ➡ tup
Axes.set_ylabel(self, ylabel, fontdict=None, labelpad=None, *, loc=None, **kwargs)
Axes.set_ylim(self, bottom=None, top=None, emit=True, auto=False, *, ymin=None, ymax=None) ➡ tup
Axes.set_yticks(self, ticks, *, minor=False) ➡ tup
```

Linestyle		Marker						Color		
"-"	"solid"	"."	•	"o"	●	"v"	▼	"b","blue"	"g","green"	"r","red"
":"	"dotted"	"s"	■	"p"	⬟	">"	▶	"c","cyan"	"m","magenta"	"k","black"
"--"	"dashed"	"*"	★	"d","D"	◆	"^"	▲	"w","white"	"y","yellow"	"#0F0F0F"
"-."	"dashdot"	"x","X"	×	"+","P"	+	"<"	◀	(0.0, 0.5, 1.0)		